

Task 5

Security Lab

Overview

This task focuses on implementing Zero Trust architecture which involves transit encryption using HTTP and TLS, mutual TLS handshakes between services and users, and user authentication and authorization using JWT (JSON Web Token) access tokens. A root certificate authority (CA), server certificate authority, and rest service authority are generated where server CA and rest service CA are leaf certificates which are then signed by the root CA. A JWT access token is generated with a limited time period to validate access to each of these services. Threat simulations with the wrong CA and invalid JWT are done to demonstrate failure/protection modes.

Github Repository: <https://github.com/bavanmendon/dsa-practicals/tree/master>

Building Certificate Authority

A root key 'ca.key' and certificate 'ca.crt' are generated using openssl alongside leaf certificate keys for server 'grpc-service.key' and REST service 'rest-service.key'. The leaf certificates issue certificate signing requests 'grpc-service.csr' and 'rest-service.csr'. These are then signed by the root CA to generate leaf certificates 'grpc-service.crt' and 'rest-service.crt'. New directories 'security/certs/' are created to store these certificates and keys. The generated keys are private keys, and hence they are added to '.gitignore', which prevents them from being uploaded to the remote repository to maintain privacy. 'ca.crt' is imported into the trust store in the user's browser.

Command: `openssl x509 -text -noout -in rest-service.crt`

The above command is used to inspect the certificate and verify SAN (Subject Alternative Name) and validity dates.

Traefik Configuration

Configuration files 'traefik.yml' and 'dynamic.yml' are created in the directory '/security'. Traefik automatically discovers running containers on specified Docker sockets and routes all HTTP/HTTPS traffic. It is setup to listen on port 443 as it's a universally standardized port for HTTP/HTTPS traffic. Since the user is sending requests to the REST service, the loadbalancer routes the incoming request to port 5000, where the REST service is hosted. Traefik presents certificates to browsers as 'ca.crt' is imported into the user's browser. However, the REST service certificate is also accepted since it was generated using a signature from the root certificate 'ca.crt'. Traefik ensures all the traffic from the user's browser is encrypted in transit using certificates signed by our private CA. This is hosted on port 8080.

Keycloak Setup

Realm is an isolated security domain that acts as an identity provider. It is hosted on port 8081, which gives a login page. The developer can log in as 'admin' to create or remove realms. In the provided 'real-export.json' file, the username and password are setup as "alice@example.com" and "secret" respectively. This file also creates the realm 'DSA Lab Realm'. When the user wants to use the application, they login to keycloak and it generates a limited time JWT which is signed by a private cryptographic key defined in 'real-export.json'. This JWT contains user data like usernames, roles, and other relevant data as per configuration. This JWT is appended with each request from the user, and the REST service verifies the identity using this access token to be able to perform the required operation.

Command:

```
export KC="http://localhost:8080/realms/dsa-lab/protocol/openid-connect"
curl -s -d "client_id=rest-client" -d "grant_type=password" \
  -d "username=alice@example.com" -d "password=secret" \
  "$KC/token" | jq -r .access_token > token.jwt
```

The above command is used to login to Keycloak, which gives the user the access token. This token is also pulled by the REST service for verification.

Mutual TLS: REST and gRPC Modifications

Before every request, the identity of the request sender and its authenticity need to be checked and verified. Since Keycloak issues the JWT for the relevant Realm (dsa-lab), the REST service is modified to check for the Authorization Bearer token for every incoming request, which is provided by Keycloak per user. If the token does not match, the request is rejected. Otherwise, the intended operation is performed.

Example request:

```
curl -k -H "Authorization: Bearer $(cat token.jwt)" https://localhost/api/items
```

In the above command, the generated access token is sent alongside the request to prove the user's identity.

CA certificates and private keys are imported into both REST and gRPC services, as these credentials are used to open secure ports for both these services to be hosted. These credentials are used for a mutual handshake between internal services to be able to encrypt communication with each other.

Docker Compose Modifications

Services from Keycloak and Gateway (Traefik) are added on ports 8081 and 8080, respectively, with relevant file and directory mounting.

Control Flow

- User logs into Keycloak.
- Keycloak generates a JWT based on user data and the registered realm.
- The user uses this JWT while sending an HTTP request.
- This request is routed via Traefik from port 443 to the REST service on port 5000.
- The REST service checks for a JWT match before processing the request. If it's a match, the operation is performed by interacting with the gRPC service
- The REST service credentials and gRPC service credentials are checked using mTLS using leaf certificates and private keys generated for both. Both the leaf certificates were created by signature from the root certificate. Hence, handshaking happens.
- The gRPC service interacts with MongoDB to retrieve requested items and sends it back to the REST service, which sends the response to the user.

Results and Failure Simulation

Build, start and check running containers using a compose file

The orchestrated containers are checked using the following command:

`docker compose ps` // List composed containers

```
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/security-lab$ docker compose up -d
[+] up 10/18
✓ Network security-lab-network Created 0.0s
✓ Container security-lab-gateway-1 Created 0.1s
✓ Container security-lab-keycloak-1 Created 0.1s
✓ Container mongodb Created 0.1s
✓ Container security-lab-jaeger-1 Created 0.1s
✓ Container security-lab-otel-collector-1 Created 0.1s
✓ Container prometheus Created 0.1s
✓ Container grpc-service Created 0.0s
✓ Container grafana Created 0.0s
✓ Container rest-service Created 0.0s
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/security-lab$ docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE    CREATED     STATUS     PORTS
grafana                             grafana/grafana:10.4.2             "/run.sh"                             grafana    12 seconds ago Up 11 seconds 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
grpc-service                        security-lab-grpc-service           "python3 -u server.py"               grpc-service 12 seconds ago Up 11 seconds 0.0.0.0:9103->9103/tcp, [::]:9103->9103/tcp, 0.0.0.0:50051->50051/tcp, [::]:50051->50051/tcp
mongodb                             mongo:latest                       "docker-entrypoint.s..."            mongodb    12 seconds ago Up 11 seconds 0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp
prometheus                         prom/prometheus:latest             "/bin/prometheus --c..."            prometheus 12 seconds ago Up 11 seconds 0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp
rest-service                       security-lab-rest-service           "python3 app.py"                     rest-service 12 seconds ago Up 10 seconds 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
security-lab-gateway-1             traefik:v3.0.1                    "/entrypoint.sh --en..."            gateway    12 seconds ago Up 11 seconds 0.0.0.0:443->443/tcp, [::]:443->443/tcp, 80/tcp, 0.0.0.0:80->80/tcp, [::]:80->80/tcp
security-lab-jaeger-1              jaegertracing/all-in-one:1.57      "/go/bin/all-in-one..."            jaeger     12 seconds ago Up 11 seconds 4318/tcp, 5775/udp, 5778/tcp, 0.0.0.0:14250->14250/tcp, [::]:14250->14250/tcp, 0.0.0.0:14268->14268/tcp, 9411/tcp, 6831-6832/udp, 0.0.0.0:16686->16686/tcp, [::]:16686->16686/tcp, 0.0.0.0:43177->43177/tcp, [::]:43177->43177/tcp
security-lab-keycloak-1            quay.io/keycloak/keycloak:26.2.5   "/opt/keycloak/bin/k..."            keycloak   12 seconds ago Up 11 seconds 8443/tcp, 9080/tcp, 0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
security-lab-otel-collector-1      otel/opentelemetry-collector-contrib:latest "/otelcol-contrib ..."            otel-collector 12 seconds ago Up 11 seconds 0.0.0.0:4317-4318->4317-4318/tcp, [::]:4317-4318->4317-4318/tcp
```

List of generated root, REST, and gRPC certificates, keys, and certificate signature requests

```
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/security-lab/security/certs$ ls
ca.crt  grpc-service.crt  grpc-service.key  rest-service.csr  san.cnf
ca.key  grpc-service.csr  rest-service.crt  rest-service.key
```

Inspection of the REST certificate by SAN and date validity verification

The inspection verified the 'Subject Alternate Name' and validity dates of these certificates. The root certificate is created with a validity of 365 days, and the leaf certificates are created with a validity of 180 days.

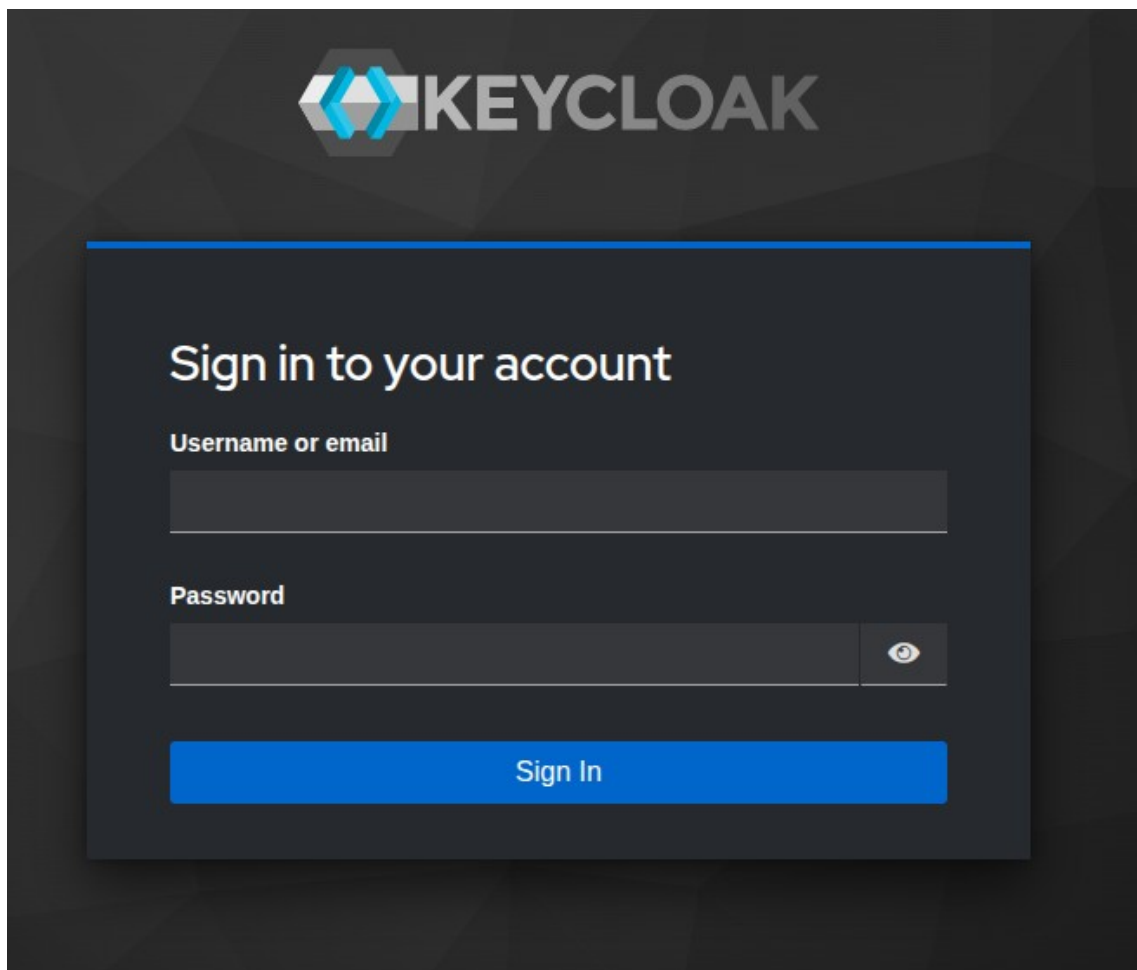
```
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/security-lab/security/certs$ openssl x509 -text -noout -in rest-service.crt
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            58:54:49:65:f4:3a:90:b2:46:86:59:fa:fc:c9:c8:46:c8:82:b6:2b
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: C = DE, O = FH-Anhalt, CN = DSA-Lab-CA
        Validity
            Not Before: Feb 26 11:52:59 2026 GMT
            Not After : Aug 25 11:52:59 2026 GMT
        Subject: C = DE, O = FH-Anhalt, CN = rest-service
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
            Public-Key: (2048 bit)
            Modulus:
                00:e0:aa:ed:01:db:2d:49:d0:8b:24:bd:c1:65:7e:
                b6:d5:7e:8c:23:ad:88:97:ba:c9:31:e1:94:06:c3:
                0a:9a:bc:bc:6b:67:bb:84:15:44:d8:41:7e:21:da:
                32:fe:2a:af:59:6a:fb:96:06:0a:f2:4d:0b:42:21:
                6c:00:7f:7d:c5:98:9a:82:11:32:b2:9e:75:8e:6a:
                4b:d7:e2:67:11:14:6a:f1:8a:1f:22:2d:b9:bb:ea:
                24:2f:7b:95:f9:23:1e:07:65:02:43:56:16:c2:5b:
                77:5b:cb:8f:ff:01:86:1d:7b:21:12:c0:69:04:29:
                cf:90:9f:1d:14:20:d4:e7:20:39:9f:3e:b0:be:ed:
                18:44:ba:2a:3a:c9:12:5b:e8:3a:24:5e:29:14:da:
                56:5c:9e:a5:06:e1:49:83:6c:cc:3c:ae:40:52:1b:
                e6:ef:11:8c:d2:ff:48:8d:bf:90:00:04:67:77:cc:
                5e:7a:b2:bc:b8:db:88:4a:34:87:fe:f2:74:a1:93:
                2f:eb:39:c8:fe:ce:98:8d:20:9e:1a:4c:4a:df:0d:
                73:bc:02:70:a4:93:79:1b:f8:70:84:b2:8b:b8:b8:
                30:84:57:f1:6a:e5:3a:bd:78:ab:40:af:ae:55:38:
                43:74:44:c0:32:ac:c9:6e:11:41:00:2a:75:76:8e:
                79:b9
            Exponent: 65537 (0x10001)
        X509v3 extensions:
            X509v3 Subject Alternative Name:
                DNS:localhost, DNS:rest-service
            X509v3 Subject Key Identifier:
                09:0B:5C:3C:AA:BF:77:A2:98:CA:7F:10:07:55:84:10:C4:0E:44:93
            X509v3 Authority Key Identifier:
                74:F4:16:E8:6B:1D:98:75:F2:37:E7:9D:6F:C2:11:1A:01:09:81:D2
```


Signature Algorithm: sha256WithRSAEncryption

Signature Value:

7f:0a:3d:b0:45:71:1a:74:92:72:a5:a4:79:ac:94:7e:7b:b7:
97:bc:56:f6:63:8d:30:14:e0:f2:ea:ad:1c:dd:5b:db:3f:8d:
f1:c3:b2:28:fd:90:49:72:5b:30:8e:27:dd:20:0e:1b:ce:5e:
75:4a:f7:7c:25:c5:e2:05:db:aa:a0:5c:0c:0a:de:52:2b:95:
b9:07:6d:44:d2:4d:2c:91:b5:dd:a3:84:58:24:f9:f4:59:13:
8e:89:a5:c6:f0:b8:e2:08:ce:49:99:c1:89:d2:83:a1:5e:29:
cc:1a:4b:d2:39:73:49:2c:04:e5:1e:a0:95:45:ac:3c:16:bf:
31:a9:7e:01:e0:76:0d:d1:2b:03:a8:67:41:11:67:d6:c7:02:
4b:f0:a1:e2:df:ce:79:70:ad:7d:fa:fd:61:d1:62:61:b3:05:
44:32:37:d3:02:34:3e:cd:dc:d9:3d:70:bd:84:09:f2:de:3f:
7f:cc:4a:c5:07:3f:56:7b:8e:a5:cd:8a:c2:33:a1:38:e7:43:
fe:3f:cc:0a:95:95:86:61:fd:f6:75:82:47:b9:a3:50:3f:fe:
3c:cd:c5:44:9f:80:26:6a:75:be:9f:34:ec:6a:5c:69:d9:b7:
bf:02:69:5e:b4:2f:d0:90:48:7f:4a:27:74:0c:1d:36:d9:60:
4f:a6:f8:8c:be:65:e6:73:ec:92:a5:d5:87:af:55:9b:c4:cc:
50:6b:27:16:b6:5e:d8:a3:75:53:cd:9a:86:07:8b:72:9f:c1:
a5:e9:5c:02:58:66:f2:bf:41:dd:e3:42:e6:4c:85:75:7d:39:
10:ae:38:31:90:3c:08:8a:d5:0d:6c:3c:61:c8:d2:5a:10:49:
1c:49:ed:65:68:d3:08:f9:72:c0:b0:b1:54:05:60:f7:1a:12:
b6:f5:59:1c:bf:0e:06:0e:db:9d:67:54:5b:e4:4c:33:b2:4b:
da:fc:1f:79:0a:6d:1e:27:fa:1c:bb:9c:55:26:b2:c1:50:2d:
8f:fc:5d:dc:c5:57:d9:1b:74:31:56:40:24:07:83:b7:c0:29:
af:d5:e6:0a:71:4a:d1:3f:41:33:f2:99:02:c4:b3:83:17:10:
08:2f:36:b7:12:34:12:a4:14:d7:cb:aa:68:6b:1f:a0:b1:c4:
48:0e:83:f5:d7:3a:c9:dd:f5:29:40:b0:65:44:12:a1:2f:64:
50:77:9c:40:07:b0:a1:30:4c:fe:bf:a8:90:3a:44:78:b3:b6:
94:60:48:b4:64:c7:1f:b9:46:f4:93:d9:fb:3c:1b:e3:3b:3d:
58:ac:c0:12:7a:b8:e9:8d:c3:10:27:73:7a:26:f0:52:e9:d2:

Keycloak Login



The image shows the Keycloak login interface. At the top, the Keycloak logo is displayed. Below it, a dark gray box contains the login form. The form has a title 'Sign in to your account'. It includes two input fields: 'Username or email' and 'Password'. The 'Password' field has a toggle icon (an eye) to the right of it. At the bottom of the form is a blue 'Sign In' button.

KEYCLOAK

Sign in to your account

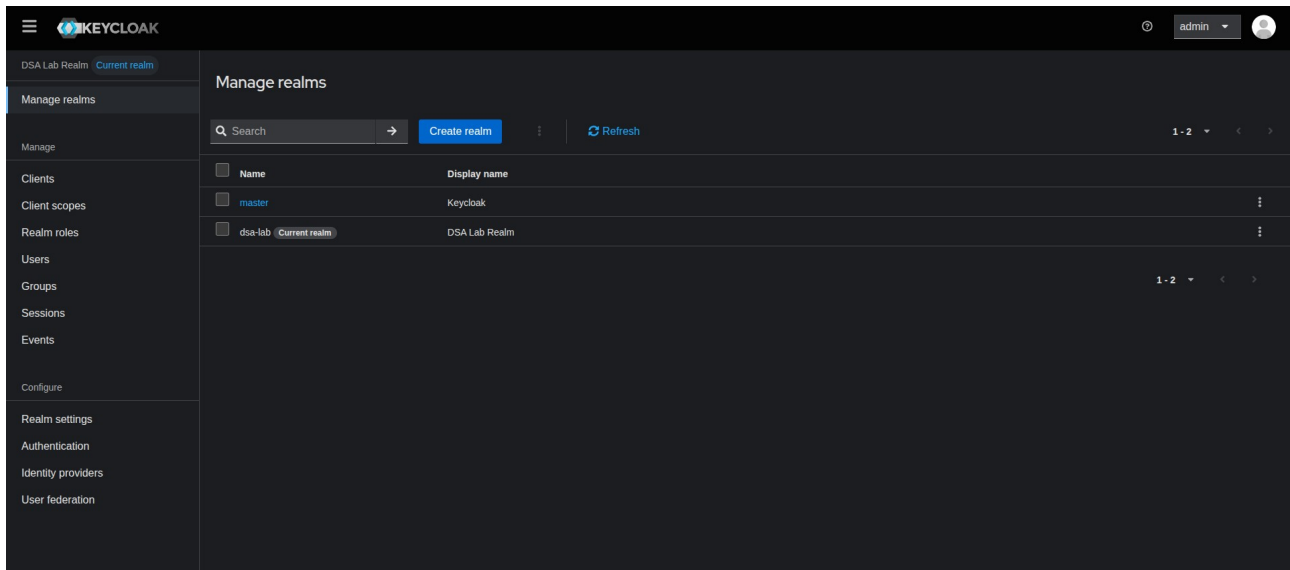
Username or email

Password

Sign In

Created Realm: DSA Lab Realm

This realm is created using the file ‘real-export.json’. This file also registers access to the username “[alice@example.com](#)” with the password “secret”.



JWT Access Token generation

The JWT token is generated by logging into Keycloak using username “[alice@example.com](#)” and password “secret”, and stored in ‘token.jwt’.

[illegible]

User Request example without JWT

Since the request was not sent with the JWT access token, the REST service successfully prevents the requested operation.

```
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/sec
urity-lab$ curl -k https://localhost/api/items
<!doctype html>
<html lang=en>
<title>401 Unauthorized</title>
<h1>Unauthorized</h1>
<p>missing Bearer token</p>
```

User Request example with JWT

Since the request was sent with the JWT access token, the REST service performs the operation.

```
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/secure-api$ curl -k -X POST https://localhost/api/items -H "Authorization: Bearer $(cat token.jwt)" -H "Content-Type: application/json" -d '{"id": 1, "name": "Bavan"}'
{"id":1,"message":"Item created","name":"Bavan"}
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/secure-api$ curl -k -X GET https://localhost/api/items -H "Authorization: Bearer $(cat token.jwt)"
[{"id":1,"name":"Bavan"}]
```


MTLS Test

The private key value is input as NULL in the REST service, which causes the handshake between the REST and gRPC service to fail.

```
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/security-lab/security/certs$ docker compose down rest-service
[+] down 2/2
  ✓ Container rest-service Removed 10.4s
  ! Network security_lab_app-network Resource is still in use 0.0s
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/security-lab/security/certs$ cd ..
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/security-lab/security$ cd ..
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/security-lab$ cd rest-service/
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/security-lab/rest-service$ nano app.py
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/security-lab/rest-service$ docker compose build rest-service
[+] Building 1.2s (12/12) FINISHED
=> [internal] load local bake definitions 0.0s
=> => reading from stdin 1.34kB 0.0s
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 200B 0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 1.0s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f75559 0.0s
=> [internal] load build context 0.0s
=> => transferring context: 11.30kB 0.0s
=> CACHED [2/5] WORKDIR /app 0.0s
=> CACHED [3/5] COPY requirements.txt . 0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> [5/5] COPY . . 0.0s
=> exporting to image 0.0s
=> => exporting layers 0.0s
=> => writing image sha256:10dc591d22b0ea005747047b85be98ecf944fdf89b8f05177ea8295d70b45e6a 0.0s
=> => naming to docker.io/library/security-lab-rest-service 0.0s
=> resolving provenance for metadata file 0.0s
[+] build 1/2
  ✓ Image security-lab-rest-service Built 1.3s
  :: Image security-lab-grpc-service Building 1.3s
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/security-lab/rest-service$ docker compose up -d rest-service
[+] up 3/3
  ✓ Container mongodb Running 0.0s
  ✓ Container grpc-service Running 0.0s
  ✓ Container rest-service Created 0.0s
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/security-lab/rest-service$ curl -s -d "client_id=rest-client" -d "grant_type=password" -d "username=alice@example.com" -d "password=secret" http://localhost:8081/realms/dsa-lab/protocol/openid-connect/token | jq -r .access_token > token.jwt
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/security-lab/rest-service$ curl -k -H "Authorization: Bearer $(cat token.jwt)" https://localhost/api/items
404 page not found
```

The private key is reinstated in the code again, and the handshake now works fine again.

```

bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/security-lab/rest-service$ docker compose down rest-service
[+] down 2/2
✓ Container rest-service Removed 0.0s
! Network security_lab_app-network Resource is still in use 0.0s
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/security-lab/rest-service$ nano app.py
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/security-lab/rest-service$ docker compose build rest-service
[+] Building 1.0s (12/12) FINISHED
=> [internal] load local bake definitions 0.0s
=> => reading from stdin 1.34kB 0.0s
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 200B 0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 0.8s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f75559 0.0s
=> [internal] load build context 0.0s
=> => transferring context: 12.48kB 0.0s
=> CACHED [2/5] WORKDIR /app 0.0s
=> CACHED [3/5] COPY requirements.txt 0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt 0.0s
=> [5/5] COPY . 0.1s
=> exporting to image 0.0s
=> => exporting layers 0.0s
=> => writing image sha256:40746a9bcc4a599875acf3c58c54de3e3eaa9a872da9cd4ba5277cb39132dc7f 0.0s
=> => naming to docker.io/library/security-lab-rest-service 0.0s
=> resolving provenance for metadata file 0.0s
[+] build 1/2
.. Image security-lab-grpc-service Building 1.1s
✓ Image security-lab-rest-service Built 1.1s
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/security-lab/rest-service$ docker compose up -d rest-service
[+] up 3/3
✓ Container mongodb Running 0.0s
✓ Container grpc-service Running 0.0s
✓ Container rest-service Created 0.0s
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/security-lab/rest-service$ curl -k -H "Authorization: Bearer $(cat token.jwt)" https://localhost/api/items
[{"id":1,"name":"Bavan"}]

```

Response when JWT expires

Since the validity of the access token has expired the user request fails and a new access token is generated to perform the operation.

```

bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/security-lab$ curl -k -H "Authorization: Bearer $(cat token.jwt)" https://localhost/api/items
<!doctype html>
<html lang=en>
<title>401 Unauthorized</title>
<h1>Unauthorized</h1>
<p>invalid token: {e}</p>
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/security-lab$ curl -s -d "client_id=rest-client" -d "grant_type=password" -d "username=alice@example.com" -d "password=secret" http://localhost:8081/realms/dsa-lab/protocol/openid-connect/token | jq -r .access_token > token.jwt
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/security-lab$ curl -k -H "Authorization: Bearer $(cat token.jwt)" https://localhost/api/items
[{"id":1,"name":"Bavan"}]

```


Response when the REST certificate name is changed

The REST service certificate name is changed, which gives an error. However, on renaming the file back to its original name, the operation is performed successfully.

```
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/security-lab$ docker compose restart rest-service
[+] restart 0/1
.: Container rest-service Restarting
bavanmendon@Electrofin:~/Documents/Anhalt/distributed-software-architecture/practicals/dsa-practicals/security-lab$ curl -k -H "Authorization: Bearer $(cat token.jwt)" https://localhost/api/items
404 page not found
```

Packet Sniffing for an encrypted payload

Command: `sudo tcpdump -i lo port 443 -A` // Sniff packets on port 443

Response from <https://localhost/api/healthz> is checked, which is supposed to be 'OK'. However, the encrypted payload is unreadable.

```
bavanmendon@Electrofin:~$ sudo tcpdump -i lo port 443 -A
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on lo, link-type EN10MB (Ethernet), snapshot length 262144 bytes
15:00:37.332874 IP6 ip6-localhost.36026 > ip6-localhost.https: Flags [.], ack 2193956011, win 746, options [nop,nop,TS val 555895452 ecr 555850433], length 0
...Z...@.....Q.....(.....
15:00:37.332886 IP6 ip6-localhost.https > ip6-localhost.36026: Flags [.], ack 1, win 649, options [nop,nop,TS val 555895452 ecr 555800541], length 0
..bV...@.....Q..R.....(.....
15:00:37.780841 IP6 ip6-localhost.https > ip6-localhost.36026: Flags [.], ack 1, win 649, options [nop,nop,TS val 555895900 ecr 555800541], length 0
..bV...@.....Q..R.....(.....
15:00:37.780868 IP6 ip6-localhost.36026 > ip6-localhost.https: Flags [.], ack 1, win 746, options [nop,nop,TS val 555895900 ecr 555895452], length 0
...Z...@.....Q..R.....(.....
15:00:42.198844 IP6 ip6-localhost.36026 > ip6-localhost.https: Flags [P.], seq 1:67, ack 1, win 746, options [nop,nop,TS val 555900318 ecr 555895452], length 66
...Z..b..@.....Q..R.....(.....
15:00:42.198988 IP6 ip6-localhost.36026 > ip6-localhost.https: Flags [P.], seq 67:106, ack 1, win 746, options [nop,nop,TS val 555900318 ecr 555895452], length 39
...Z..G..@.....Q.....(.....
15:00:42.199096 IP6 ip6-localhost.https > ip6-localhost.36026: Flags [.], ack 106, win 649, options [nop,nop,TS val 555900318 ecr 555900318], length 0
..bV...@.....Q.....(.....
15:00:42.199349 IP6 ip6-localhost.https > ip6-localhost.36026: Flags [P.], seq 1:40, ack 106, win 649, options [nop,nop,TS val 555900318 ecr 555900318], length 39
..bV..G..@.....Q.....(.....
15:00:42.202912 IP6 ip6-localhost.https > ip6-localhost.36026: Flags [P.], seq 40:99, ack 106, win 649, options [nop,nop,TS val 555900322 ecr 555900318], length 59
..bV..[..@.....Q.....(.....
15:00:42.202937 IP6 ip6-localhost.https > ip6-localhost.36026: Flags [P.], seq 99:132, ack 106, win 649, options [nop,nop,TS val 555900322 ecr 555900318], length 33
..bV..A..@.....Q.....(.....
15:00:42.203040 IP6 ip6-localhost.36026 > ip6-localhost.https: Flags [.], ack 132, win 746, options [nop,nop,TS val 555900322 ecr 555900318], length 0
...Z...@.....Q.....(.....
15:00:42.205389 IP6 ip6-localhost.36026 > ip6-localhost.https: Flags [P.], seq 106:141, ack 132, win 746, options [nop,nop,TS val 555900324 ecr 555900318], length 35
...Z..G..@.....Q.....(.....
15:00:42.240823 IP6 ip6-localhost.https > ip6-localhost.36026: Flags [.], ack 141, win 649, options [nop,nop,TS val 555900366 ecr 555900324], length 0
..bV...@.....Q.....(.....
15:00:43.412887 IP6 ip6-localhost.36022 > ip6-localhost.https: Flags [.], ack 2068487568, win 746, options [nop,nop,TS val 555901532 ecr 555856378], length 0
..G...@.....Q.....(.....
15:00:43.412917 IP6 ip6-localhost.https > ip6-localhost.36022: Flags [.], ack 1, win 669, options [nop,nop,TS val 555901532 ecr 555887196], length 0
..I...@.....Q.....(.....
15:00:44.436783 IP6 ip6-localhost.https > ip6-localhost.36022: Flags [.], ack 1, win 669, options [nop,nop,TS val 555902556 ecr 555887196], length 0
..I...@.....Q.....(.....
15:00:44.436798 IP6 ip6-localhost.36022 > ip6-localhost.https: Flags [.], ack 1, win 746, options [nop,nop,TS val 555902556 ecr 555901532], length 0
..G...@.....Q.....(.....
```

Browser with green lockpad

The certificate check verification shows that the connection is secure as expected after importing 'ca.crt' into the browser's trust store.

